

HEAP TREE

* A Heap Tree is a complete binary tree that satisfies the heap property, where each ~~element~~ parent node is either greater than (or) equal to its children (Max Heap) or less than or equal to its children (Min Heap).

* It is commonly used to implement priority queues, allowing efficient insertion, deletion and retrieval of the highest - or lowest - priority element.

Characteristics of a Heap Tree.

- * It is a complete Binary Tree
- * Every level is completely filled except possibly the last level.
- * The last level is filled from left to right.
- * usually stored in an array
- * Supports efficient insertion and deletion

TYPES OF HEAP TREES:

A Heap tree is classified into two types

1. MAX Heap

2. MIN Heap

1. MAX HEAP

* In max Heap, every parent node is greater than or equal to its children

* Therefore the largest element is always stored at the root.

Eg: Constructing a max Heap

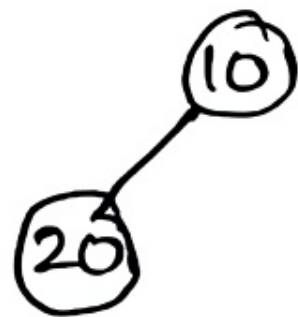
Given elements: 10, 20, 15, 30, 40

Sol Insert elements one by one while maintaining the Max heap property

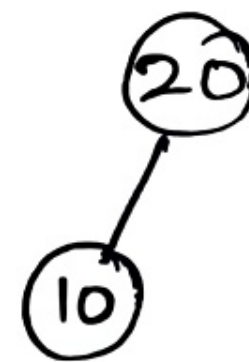
Step1: Insert 10



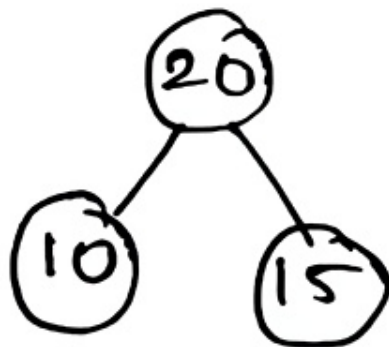
Step2: Insert 20



20 is greater than 10, So swap

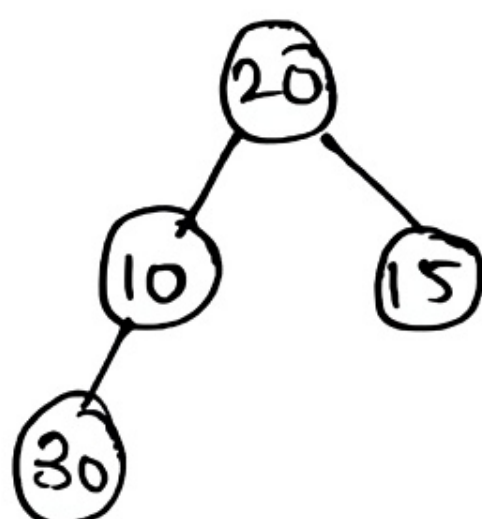


Step3: Insert 15



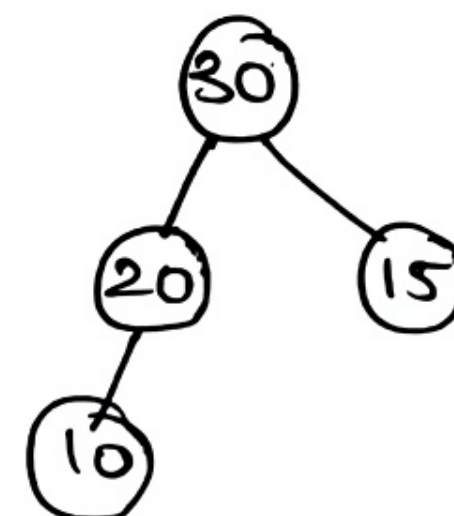
$15 < 20$, No swap

Step4: Insert 30

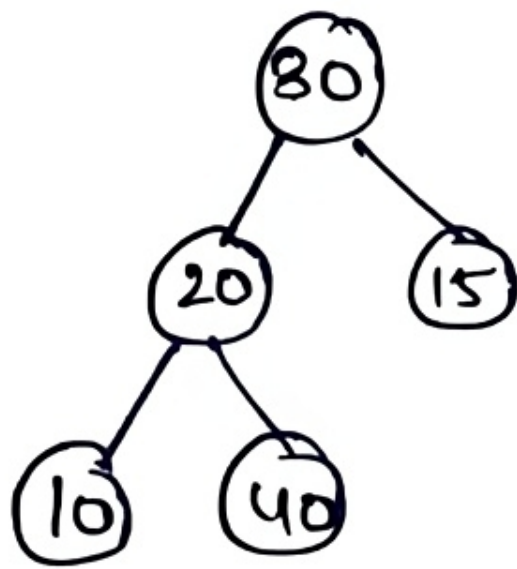


$30 > 10$, swap

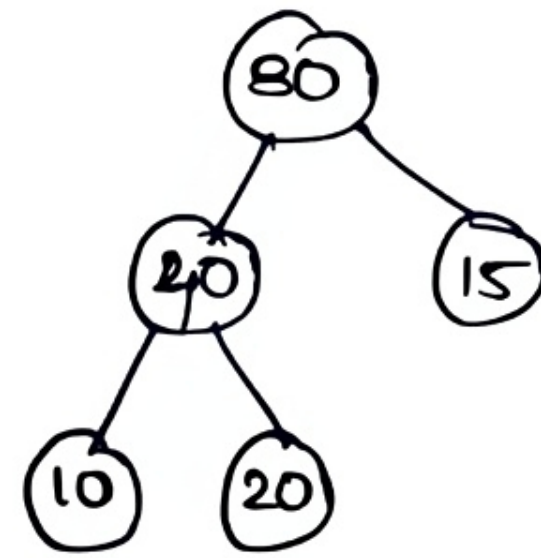
$30 > 20$, swap again



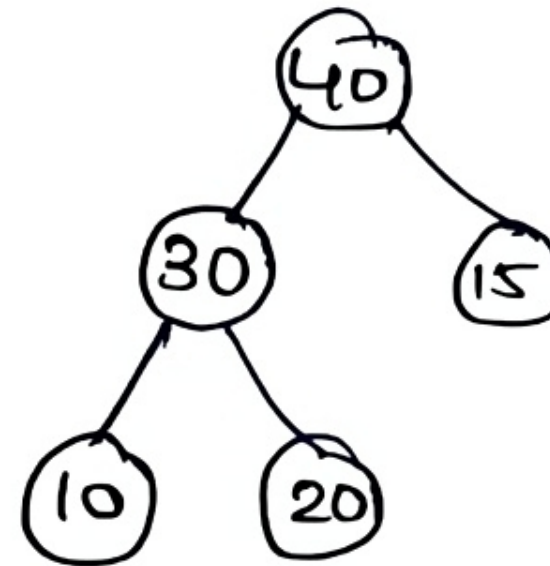
Steps: Insert 40



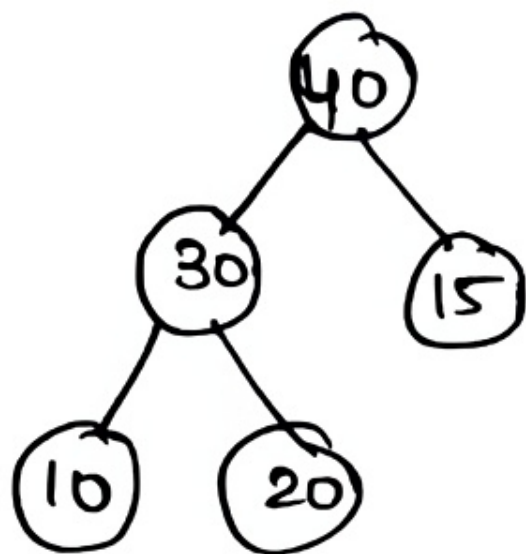
40 > 20, swap



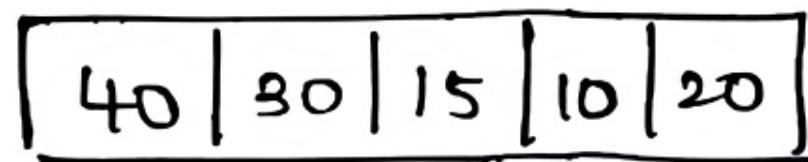
40 > 30 swap



FINAL MAX HEAP



Array Representation:



∴ It satisfies the MAX Heap property.

Characteristics of MAX Heap:

- * Root contains the largest element.
- * Parent node $\geq$ child nodes
- * used in maximum priority queues

2. MIN HEAP

- * In a Min Heap, every parent node is less than (or) equal to its children.
- * Therefore the smallest element is always stored at the root.

Characteristics of min Heap:

- * Root contains the smallest element
- * Parent node $\leq$ child nodes.
- * used in minimum priority queues.

Note: Array Representation

for an element at index i (0-based indexing):

$$\rightarrow \text{Parent} = (i-1)/2$$

$$\rightarrow \text{Left child} = 2i+1$$

$$\rightarrow \text{Right child} = 2i+2$$

for index i (1-based indexing):

$$\rightarrow \text{Parent} = i/2$$

$$\rightarrow \text{Left child} = 2*i$$

$$\rightarrow \text{Right child} = 2*i+1$$

eg: constructing a min heap for elements: 40, 30, 15, 10, 20.

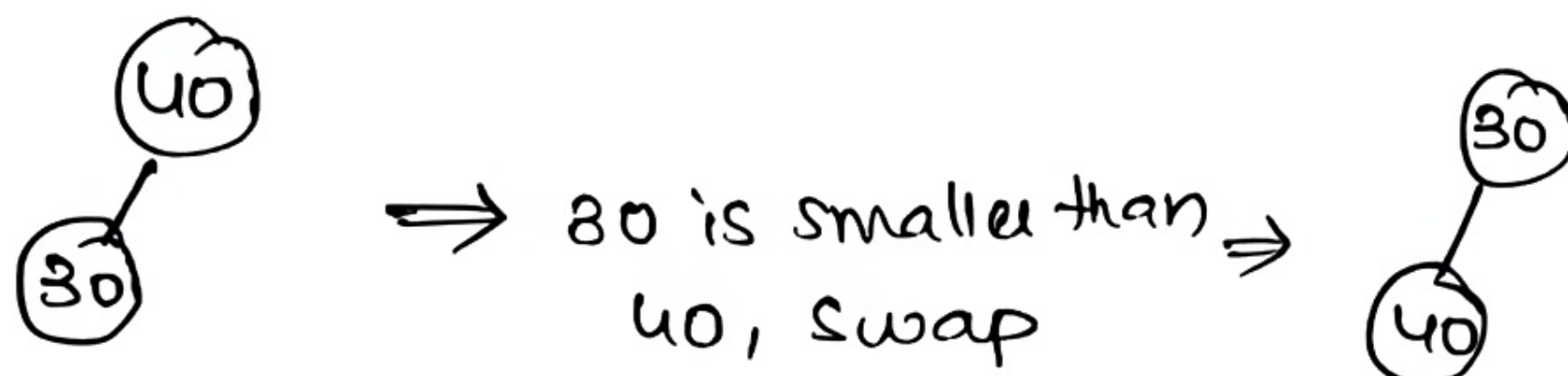
Sol Insert the elements one by one while maintaining

the min heap property (Parent $\leq$ children)

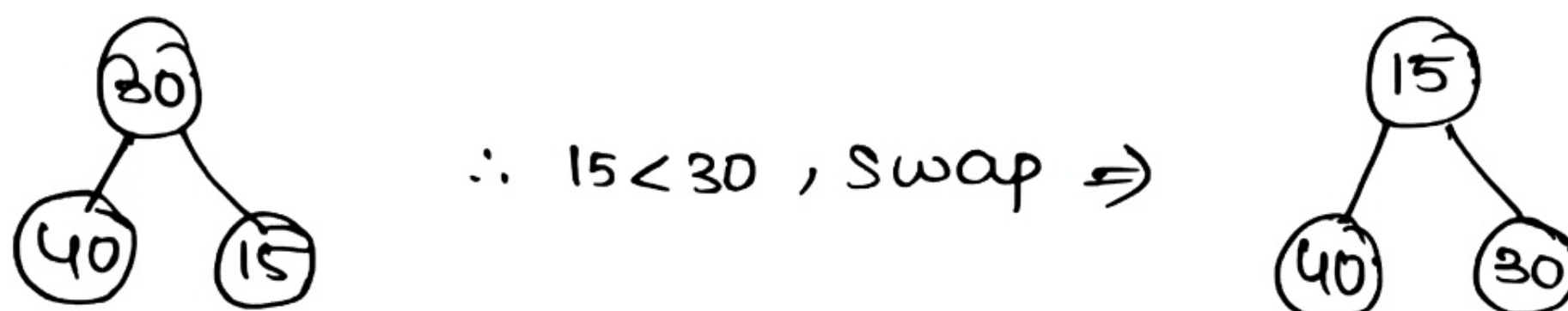
Step1: Insert 40



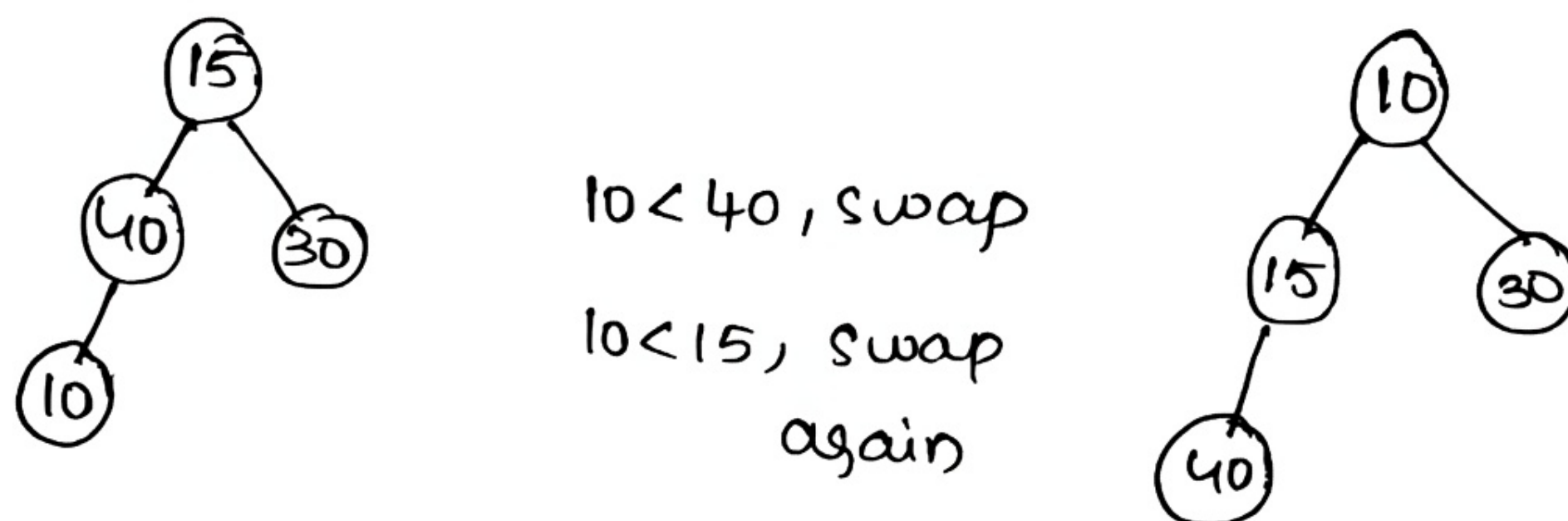
Step2: Insert 30



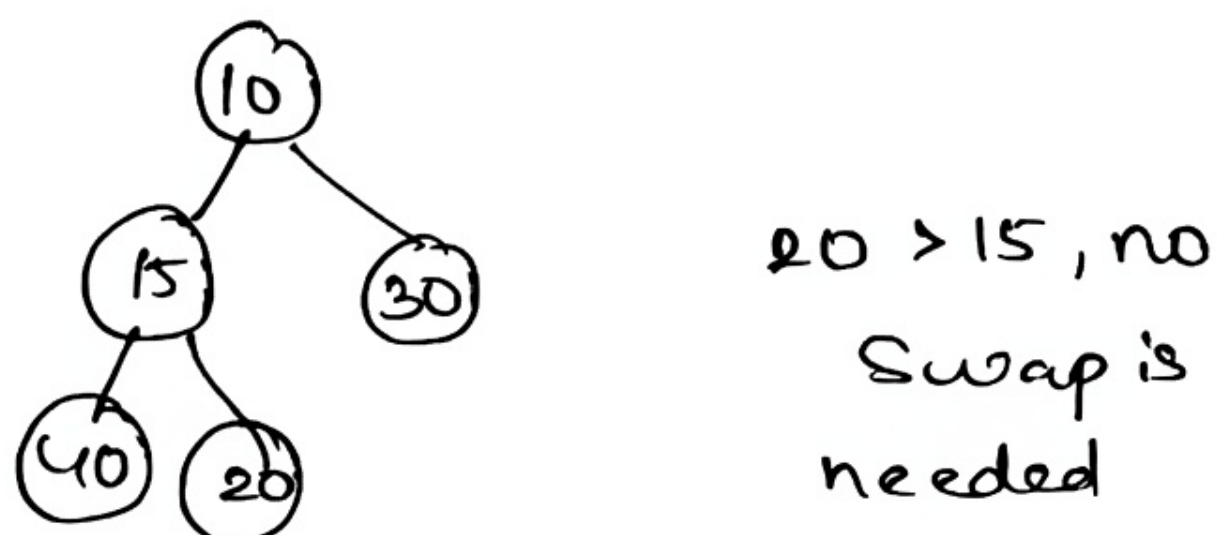
Step3: Insert 15



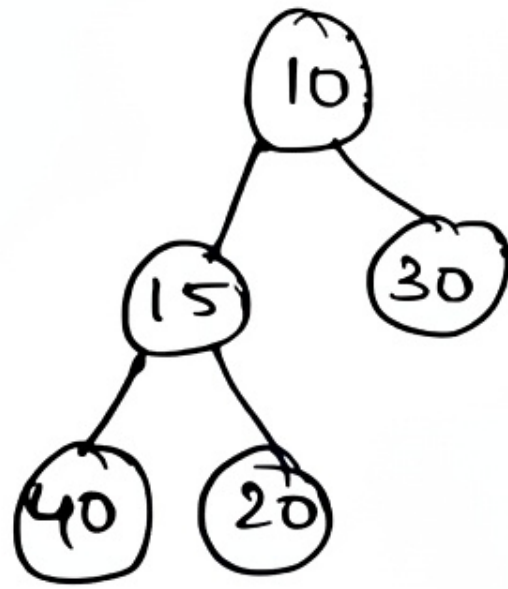
Step4: Insert 10



Step5: Insert 20



∴ Final Min Heap



Array Representation:

10	15	30	40	20
----	----	----	----	----

The main operations of a heap are:

(4)

1. Insertion
2. Deletion
3. Peek
4. Heapify
5. Build Heap

INSERTION: (Heapify up)

- * Adds a new element to the heap.
- * The element is first placed at the last position.
- * It is then moved upward (heapify up) until the heap property is restored.
- * Time complexity: $O(\log n)$

Algorithm:

1. Insert the new element at the last position in the heap.

2. Compare it with its parent.

3. If the heap property is violated:

- * Max Heap: $\text{child} > \text{parent} \rightarrow \text{Swap them.}$

- * Min Heap: $\text{child} < \text{parent} \rightarrow \text{Swap them.}$

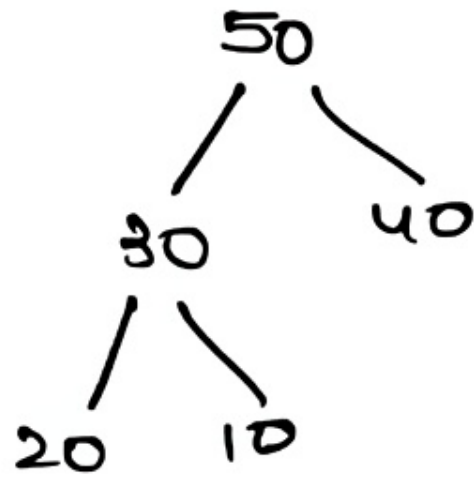
4. Repeat until:

- $\rightarrow$ The heap property is satisfied or

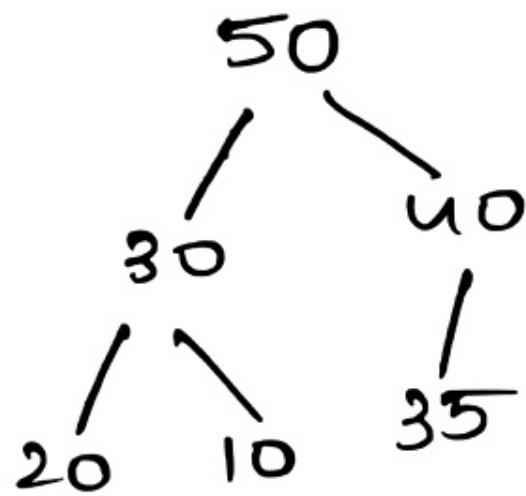
- $\rightarrow$ The element becomes the root.

Eg: (MAX HEAP)

Initial heap:



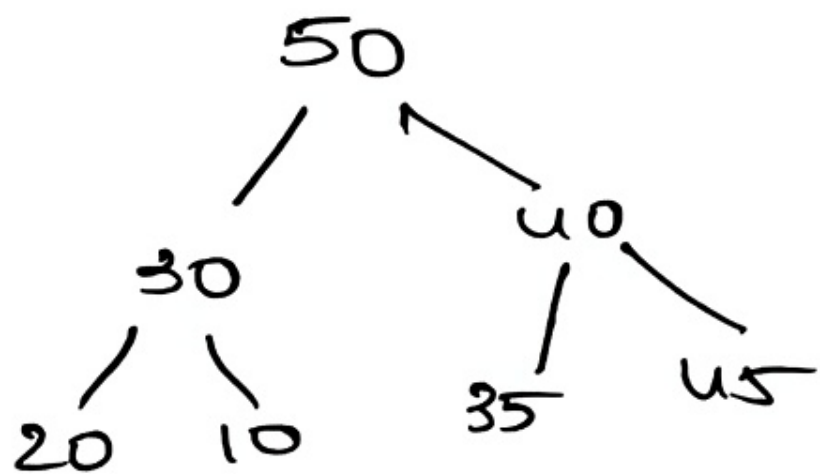
Insert 35:



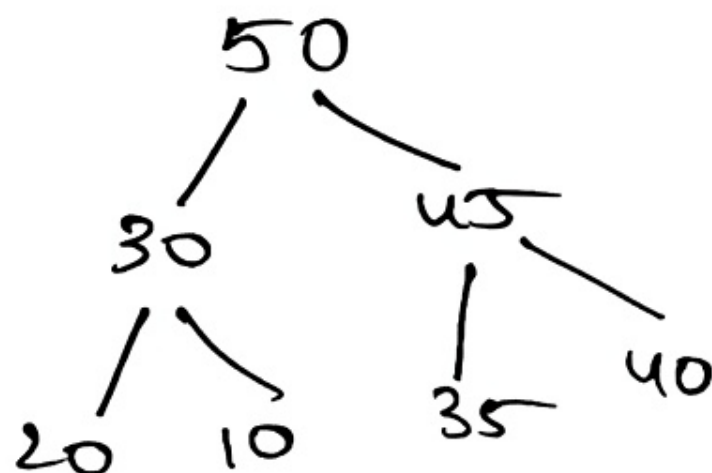
→ compare 35 with its parent 40
 $35 < 40$, so heap property satisfied.

No swap, needed

Insert 45:

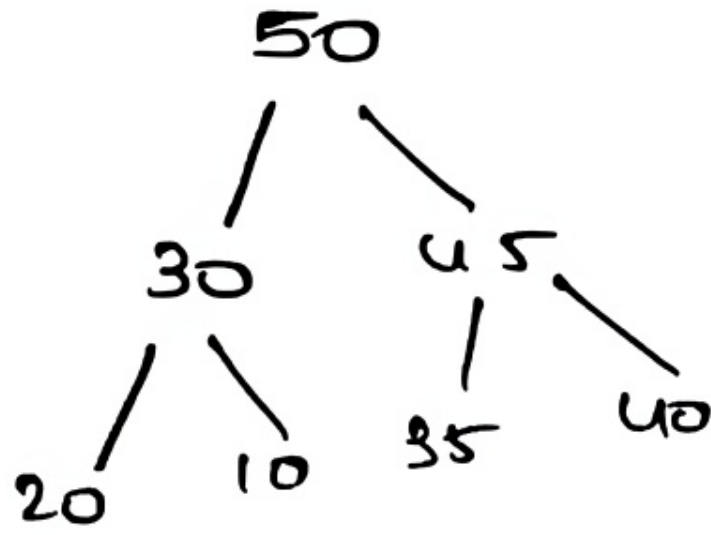


$45 > 40 \rightarrow \text{swap}$



FINAL heap:

(5)



Time complexity:

Best case: $O(1)$

Worst case: $O(\log n)$

Space complexity: $O(1)$

2. Deletion in a heap (max heap)

Deletion usually means removing the root element (the maximum element in a max heap)

Steps:

1. Delete the root element.

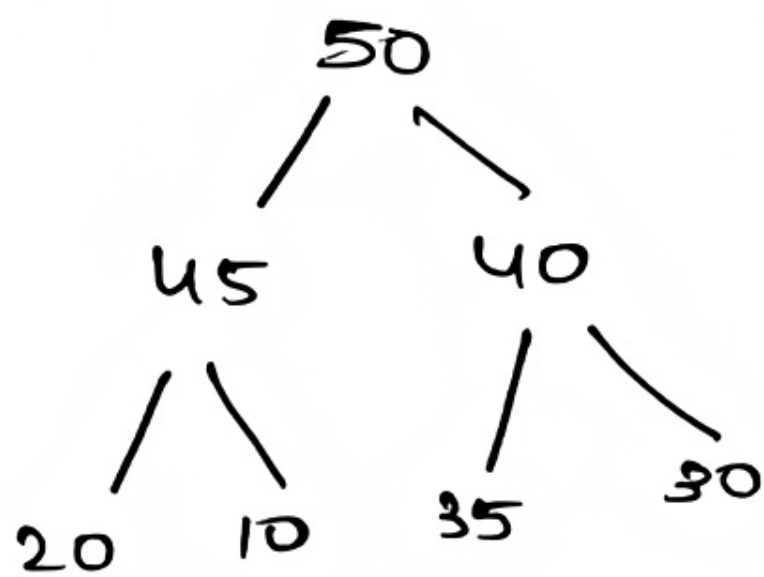
2. Move the last element in the heap to the root.

3. Remove the last element.

4. Perform heapify down to restore the heap

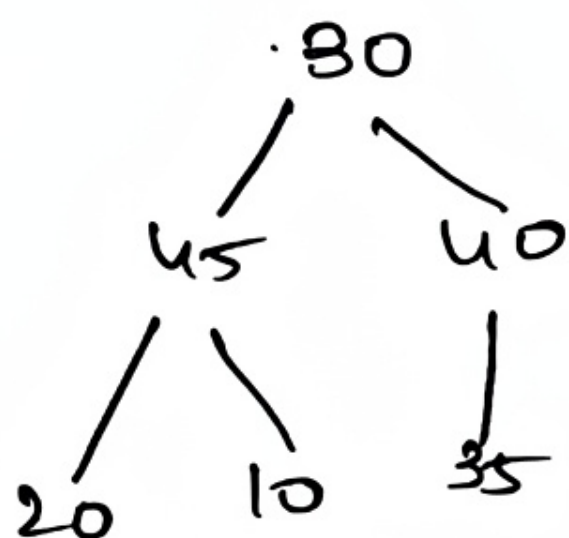
property.

Eg: Initial heap



Delete the root : 50

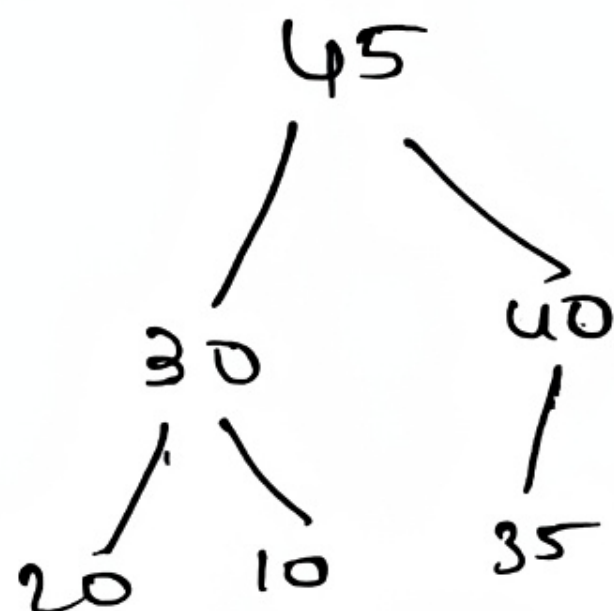
Step 2: Move the last element (30) to the root



Step 3: Heapify down

Compare 30 with children 45 & 40

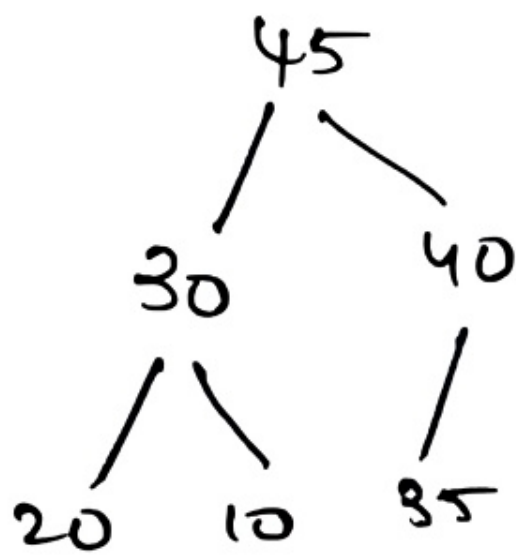
Largest child is 45 → swap



∴ compare 30 with children 20 & 10

→ 30 is larger than both, so stop.

FINAL HEAP:



Time complexity

Deletion: $O(\log n)$

Replace root: $O(1)$

Heapify down: $O(\log n)$

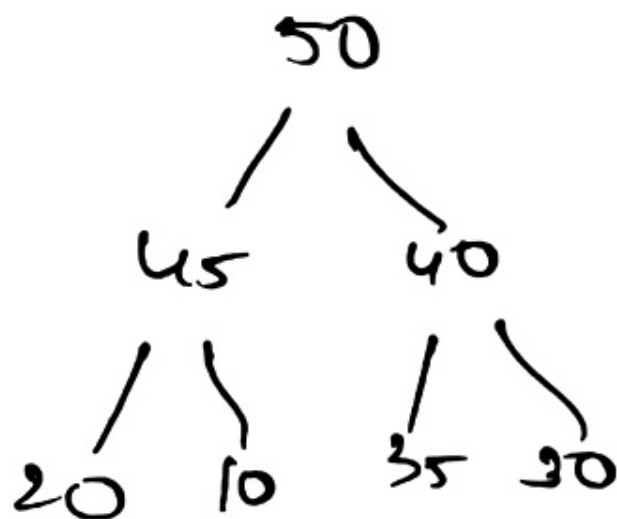
3) PEEK OPERATION:

In peek operation returns the root element of the heap without removing it.

* In a max heap, peek() returns the largest element.

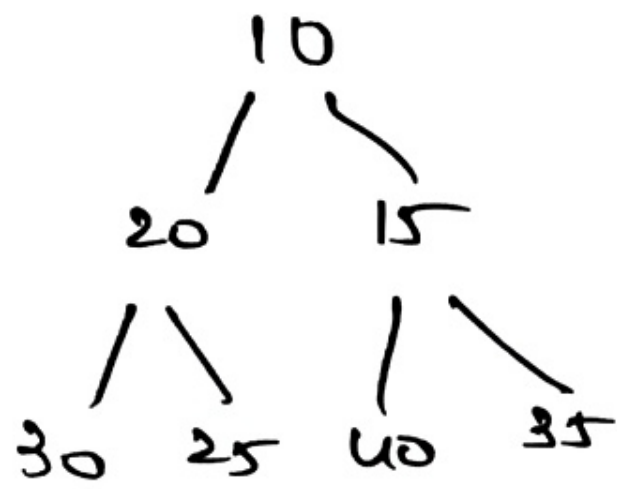
* In a min heap, peek() returns the smallest element.

Eg: (Max Heap)



• peek() returns 50

Eg. (MIN Heap)



peek() returns 10

Time complexity:

peek : $O(1)$

4) Heap sort:

Heap Sort is a sorting technique that uses a Max Heap (a binary tree where the parent is always greater than (or) equal to its children).

⇒ Heap Sort has two phases:

1. Build Max Heap from the array
2. Sort the Heap (Repeatedly Swap the root with the last element and heapify until the array is sorted).

Steps:

1. Build a Max heap from the array
2. The largest element is at the root
3. Swap the first element with the last element.

Heap sort:

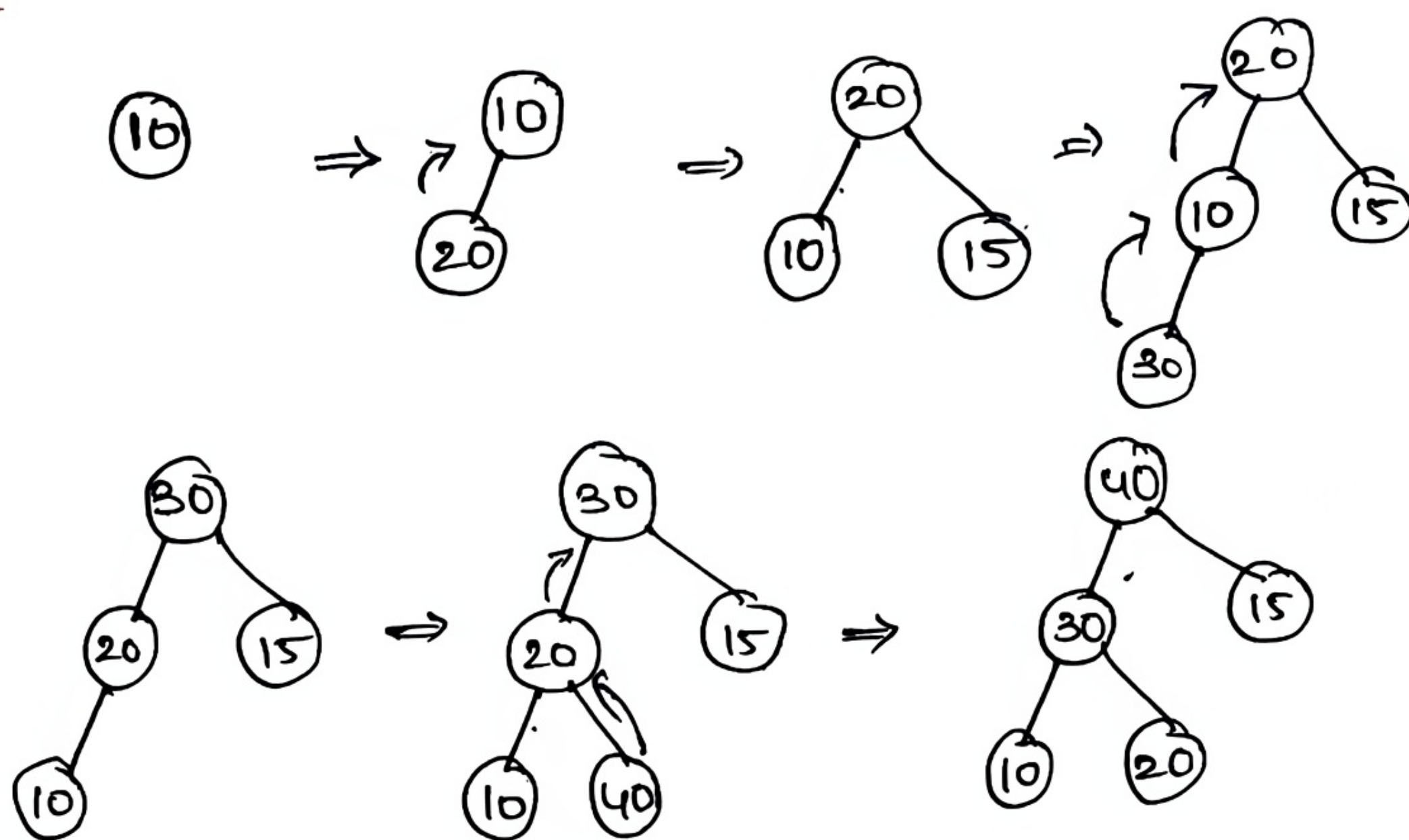
(7)

4. Reduce the heap size by one.
5. Heapify the root again to make it a max heap.
6. Repeat until all elements are sorted.

eg: Given Array:

10, 20, 15, 30, 40

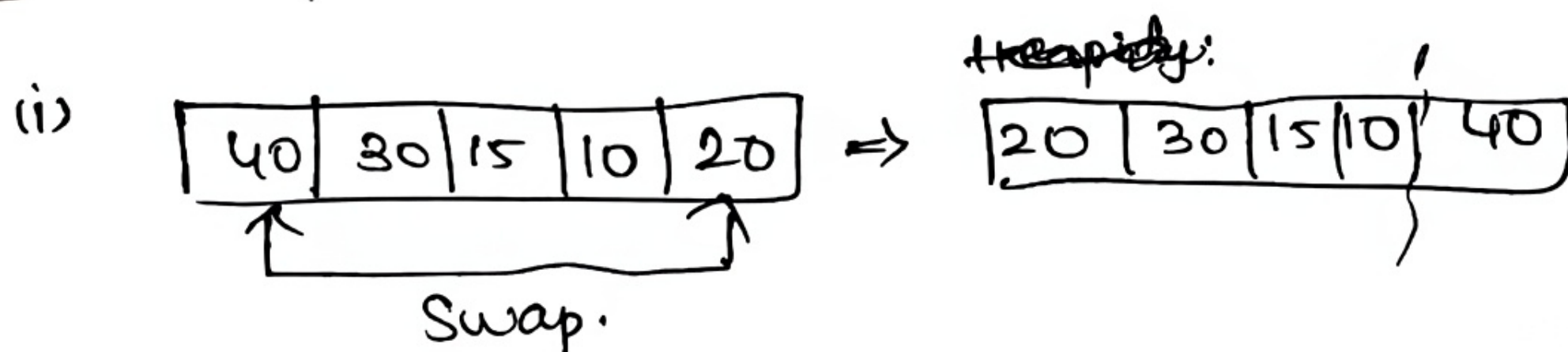
Step 1: Build max heap.

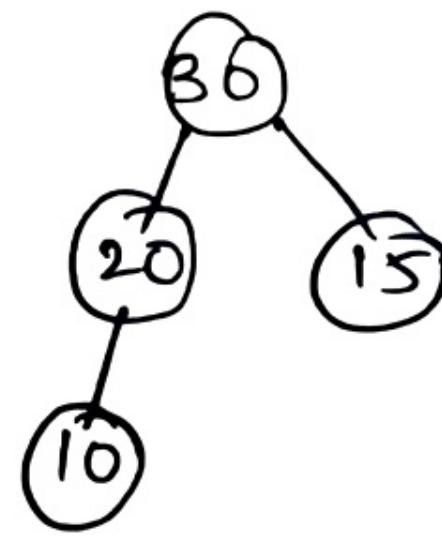
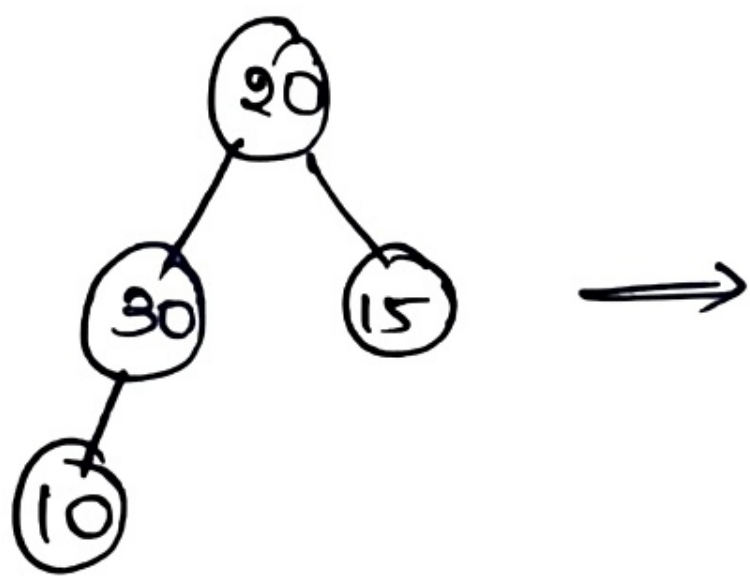


Array elements after max heap is:

40, 30, 15, 10, 20

Step 2: Swap the first element with the last element



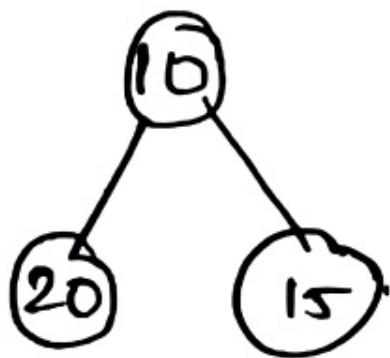


Not satisfies Heap property

∴ Array list is :

30	20	15	10	40
----	----	----	----	----

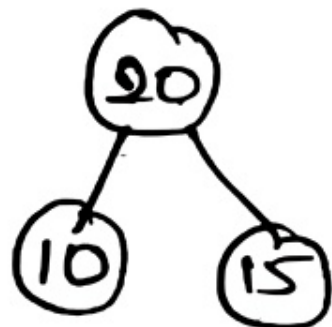
(ii) Swap 30 with 10.



∴ Array list is

10	20	15	30	40
----	----	----	----	----

∴ Tree not satisfies heap property

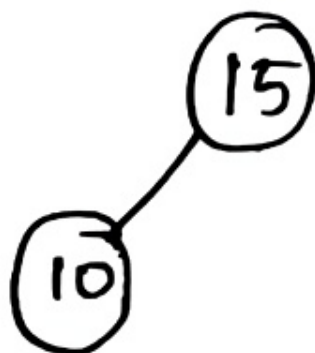


Heapify:

∴ Array list is

20	10	15	30	40
----	----	----	----	----

(iii) Swap 20 with 15



∴ Array list is

15	10	20	30	40
----	----	----	----	----

(iv) Swap 15 with 10



∴ Array list is

10	15	20	30	40
----	----	----	----	----

∴ FINAL SORTED ARRAY IS

10	15	20	30	40
----	----	----	----	----

Time complexity for Heap sort:

$$O(n \log n)$$

Space complexity: $O(1)$. (IN-PLACE SORTING)

5) Heapify OPERATION:

Heapify is the process of adjusting a binary tree so that it satisfies the heap property.

Two types of heaps:

→ MAX HEAP

→ MIN HEAP

1 1 1 1 = empty.

APPLICATIONS OF HEAP:

* Priority Queues

* Heap Sort

* Graph Algorithms:

→ Dijkstra's algorithm

→ Prim's algorithm

* Finding the k th largest / smallest element

* Merging k sorted Arrays / lists

* Operating Systems

* Memory management

CONNECTED COMPONENTS

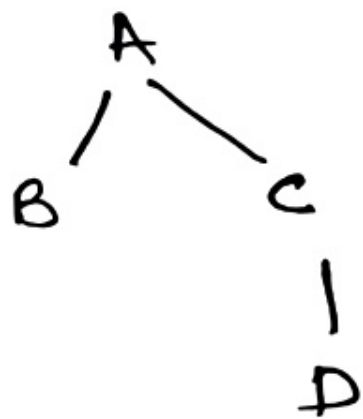
A connected component is a group of vertices in an undirected graph where:

- * Every vertex is reachable from every other vertex in that group.
- * There are no edges connecting vertices of different components.

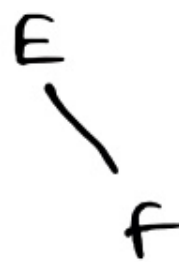
Eg:

Consider this graph:

Component 1:



Component 2:



Here: $\{A, B, C, D\}$ is one connected component.

$\{E, F\}$ is another connected component.

So the graph has 2 connected components.

FINDING CONNECTED COMPONENTS USING DFS

1. Keep all vertices initially unvisited.
2. Start DFS from any unvisited vertex.
3. All vertices reached by this DFS belong to one connected component.
4. Repeat until all vertices are visited.

5. count how many dfs calls were made

Algorithm

Count = 0

for each vertex v in graph:

If v is not visited:

DFS(v)

Count++

DFS(v)

mark v is visited

for each neighbour u of v :

If u is not visited:

DFS(u)

Process:

1. Start DFS(A)

* VISITS $\{A, B, C, D\}$

* Component Count = 1.

2. Start DFS(E)

* VISITS $\{E, F\}$

* Component Count = 2

O/p

No. of connected components = 2

Time Complexity: FOR AN ADJACENCY LIST REPRESENTATION

$O(V+E)$

* each vertex is visited once $\rightarrow O(V)$

* each edge is checked once $\rightarrow O(E)$

using BFS

Count = 0

for each vertex v :

If v is not visited:

BFS(v)

Count++

Space complexity:
 $O(V)$

Because:

- * visited array store V vertices
- * DFS recursion stack (or) BFS queue can store up to V vertices.

APPLICATIONS OF CONNECTED COMPONENTS

- * finding groups in social networks
- * Detecting isolated networks
- * Image processing (connected regions)
- * finding clusters in data
- * checking if all nodes in a n/w are reachable.

NOTE

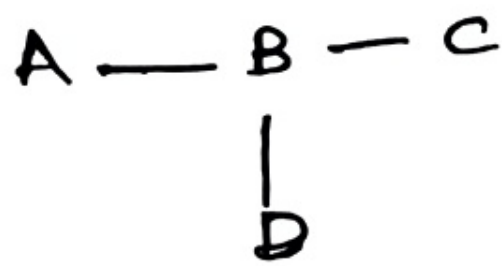
- * Connected components are defined for undirected graphs.
- * Strongly connected components are defined for directed graphs.

ARTICULATION POINT

* An articulation point is a critical vertex. If you remove it, the graph breaks into disconnected parts.

* It is also called a cut vertex.

Eg1: Graph with an articulation point.



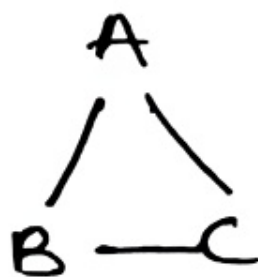
Remove B from graph, Then graph look like



∴ The graph becomes disconnected.

∴ B is an articulation point.

Eg2: Graph without an articulation point



→ If you Remove any vertex from above graph:

* Remove A → B, C are still connected through edge B-C

∴ NO vertex can disconnect the graph

∴ So this graph has no articulation points.

BICONNECTED GRAPH

(or)

NOTE: Can I still stay connected if one node fails?

BICONNECTED COMPONENT

* A biconnected component also called as 2-connected component or block.

* A biconnected component is a part of a graph where there is no single vertex whose removal can disconnect it.

(or)

* A biconnected component is a group of nodes in a graph that stays connected even if one node is removed.

* A connected graph can have one (or) more biconnected components.

* Biconnected components are separated by articulation points.

Eg:



$\{A, B, C\}$ is a biconnected component

$\{B, D\}$ is a biconnected component.

* If, you Remove B, the remaining graph is
→ Still {A, C} are still connected.

A — C. [It is part of biconnected component]

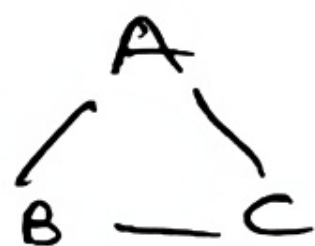
* But D, disconnected, because its only
connection was through B.

∴ A & C are connected &

D is isolated.

∴ B is an articulation point.

NOTE



form a cycle A — B — C — A
then say Biconnected component.

Applications of Biconnected components:

1. Network Reliability
2. Power Grid Analysis
3. Social Network Analysis
4. Circuit design
5. Software Engineering
6. Geographic Information systems

DFS ALGORITHM DFS & BFS SEARCHING TECHNIQUES

Algorithm DFS (Graph, $\underset{\substack{\downarrow \\ S}}{\text{start}} \Rightarrow (G, S)$
{

Let S be stack

Create an empty stack

Push start onto stack

S.push(S)

Mark S is visited

while (S is not empty)

{
 $V = S.top()$

 S.pop()

 visit(V)

 //

 // push all the neighbours of 'V' in to stack
 that are not visited.

 for each neighbour w of V in graph G

 {
 if w is not visited:

 {
 S.push(w)
 ↳ neighbour.

 mark w is visited

 }

 }

}

Time complexity: $O(V + E)$
 ↳ visiting vertex
 ↳ exploring vertex

Space complexity: $O(V)$ $\Rightarrow$ typically maintain
visited array to
avoid revisiting vertices.

BFS-ALGORITHM

Algorithm BFS (G, S)

// G is a Graph & S is Source node

{

Let Q be Queue

$Q.enqueue(S)$

// Insert S in Queue until all the neighbour
Vertices are marked

Mark S as visited

while (Q is not empty)

{ // Remove that vertex from queue, where
neighbour will be visited now

$V = Q.dequeue()$

// Process all the neighbour of V
& visit (vertex)
for all neighbours w of V in Graph G

{ if (w is not visited)

{ $Q.enqueue(w)$ // Store w in Q to further

visit its neighbour
Mark w as visited.

}
}

Time complexity of BFS $O(V + E)$

Space complexity is $O(V)$

↳ Queue: $O(V)$

↳ visited array: $O(V)$

} $O(V) + O(V) = O(V)$

RECURRENCES

A Recurrence relation is a mathematical equation that defines a problem's solution (or running time) in terms of smaller instances of the same problem.

* In DAA, recurrence relations are mainly used to find the time complexity of recursive algorithms

* Recurrence equation is an equation that defines a sequence recursively. It expresses the value of a term using the value of previous terms.

eg:
$$T(n) = T(n-1) + n \quad \text{for } n > 0$$

$$T(0) = 0$$

$\therefore T(n) = T(n-1) + n$ is a Recurrence relation.

because it defines $T(n)$ using the previous value

$$T(n-1)$$

Here $T(0) = 0 \rightarrow$ It is Initial condition (Base condition)

$\rightarrow$ This tells where the recursion stops.

$\rightarrow$ Without an Initial condition, the recurrence cannot be solved.

Note: Recurrence Relation = Relationship b/w current & previous terms.

METHODS TO SOLVE RECURRENCE EQUATIONS;

1. Substitution Method.
2. Recursion Tree Method
3. Master Theorem. Method

1. Substitution Method;

The Substitution method is a kind of method consists of two steps:

* Guess the Solution.

* Prove it using Mathematical Induction

It is also called the Induction method.

Types of Substitution method;

1. Forward Substitution (or) Direct Substitution
2. Backward substitution

1. ^{BACK}~~FORWARD~~ SUBSTITUTION

~~Backward~~ substitution means repeatedly replacing the recurrence with the next smaller term until we reach the base condition.

* Start from $T(n)$ → expand toward base case

eg: Consider a recurrence relation

$$T(n) = T(n-1) + n$$

Step 1: expand ~~backwards~~ forward

$$T(n) = T(n-1) + n \rightarrow \text{eq (1)}$$

Substitute $n = n-1$

$$\begin{aligned} T(n-1) &= T(n-1-1) + (n-1) \\ &= T(n-2) + (n-1) \rightarrow \text{eq (2)} \end{aligned}$$

Substitute eq (2) in eq (1) we get.

$$T(n) = T(n-2) + (n-1) + n \rightarrow \text{eq (3)}$$

Again:

$n = n-2$ substitute in eq (1)

$$\begin{aligned} T(n-2) &= T(n-2-1) + (n-2) \\ T(n-2) &= T(n-3) + (n-2) \rightarrow \text{eq (4)} \end{aligned}$$

Sub eq (4) in eq (3) we get.

$$\begin{aligned} T(n) &= T(n-3) + (n-2) + (n-1) + n \\ &\vdots \quad \vdots \quad \vdots \quad \vdots \end{aligned}$$

Assume After k steps

$$T(n) = T(n-k) + (n-(k-1)) + (n-(k-2)) + \dots + (n-1) + n$$

$$\therefore T(n) = T(n-k) + (n-k+1) + \dots + (n-1) + n$$

when

$$n - k = 0$$

$$k = n$$

Using base condition:

$$T(n) = T(n-n) + (n-n+1) + \dots + (n-1) + n$$

$$T(n) = T(0) + 1 + 2 + \dots + (n-1) + n$$

$$T(0) = 0$$

$$T(n) = 0 + 1 + 2 + \dots + (n-1) + n$$

~~~~~

$$\therefore \text{The formula generated } 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

By observing, generated equations can derive a formula.

$$T(n) = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$$

$$\therefore \boxed{T(n) = O(n^2)} \quad \left[ \text{we denote } T(n) \text{ in terms of big-oh notation as follows} \right]$$

## 2. FORWARD SUBSTITUTION;

\* In forward Substitution, we start from the base case (Initial condition) and calculate values for increasing  $n$ :

$$n = 1, 2, 3, \dots$$

UNIT-2 (3)

We use previous values to find the next values  $c_4$

Identify the pattern.

Eg: Given Recurrence Relation:

$$T(n) = T(n-1) + n$$

Base condition Initial condition:

$$T(0) = 0$$

Now calculate forward:

for  $n=1$ , then

$$\begin{aligned} T(1) &= T(1-1) + 1 \\ &= T(0) + 1 \\ &= 0 + 1 \end{aligned}$$

$$\boxed{T(1) = 1}$$

for  $n=2$ , then

$$\begin{aligned} T(2) &= T(2-1) + 2 \\ &= T(1) + 2 \\ &= 1 + 2 \\ T(2) &= 3 \end{aligned}$$

for  $n=3$ , then

$$\begin{aligned} T(3) &= T(3-1) + 3 \\ &= T(2) + 3 \\ T(3) &= 3 + 3 = 6 \end{aligned}$$

$$\begin{aligned}\text{for } n=4, \text{ then } T(4) &= T(4-1) + 4 \\ &= T(3) + 4 \\ &= 6 + 4\end{aligned}$$

$$T(4) = 10$$

Pattern generated / formula generates!

$$T(n) = 1 + 3 + 6 + 10 + \dots + n$$

These can be written as:

$$T(n) = \frac{1(1+1)}{2} + \frac{2(2+1)}{2} + \frac{3(3+1)}{2} + \dots + n$$

$$T(n) = \frac{n(n+1)}{2}$$

$$T(n) = \frac{n^2}{2} + n/2$$

$\therefore$  Denote  $T(n)$  in terms of big-oh notation as follows

$$\boxed{T(n) = O(n^2)}$$

## 2. Recursion Tree Method:

\* Represents the recurrence as a tree.

\* calculates the levels of recursion at each level.

Eg: Algorithm is

void a (int n)  $\Rightarrow T(n)$

{

if (n > 0)  $\Rightarrow 1$

{

printf ("%d", n);  $\rightarrow 1$

}

a(n-1); // recursive call for 'a'  $\Rightarrow T(n-1)$

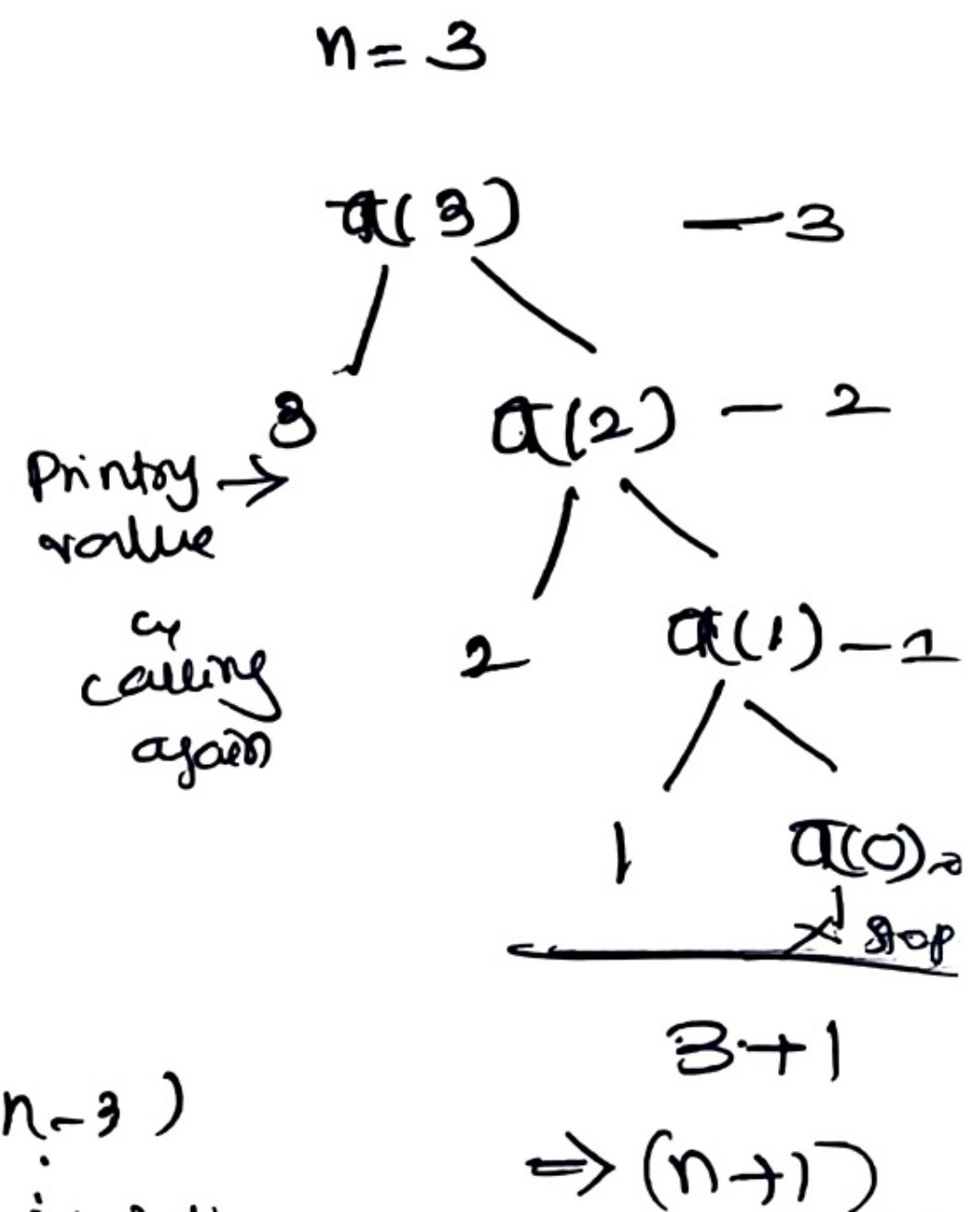
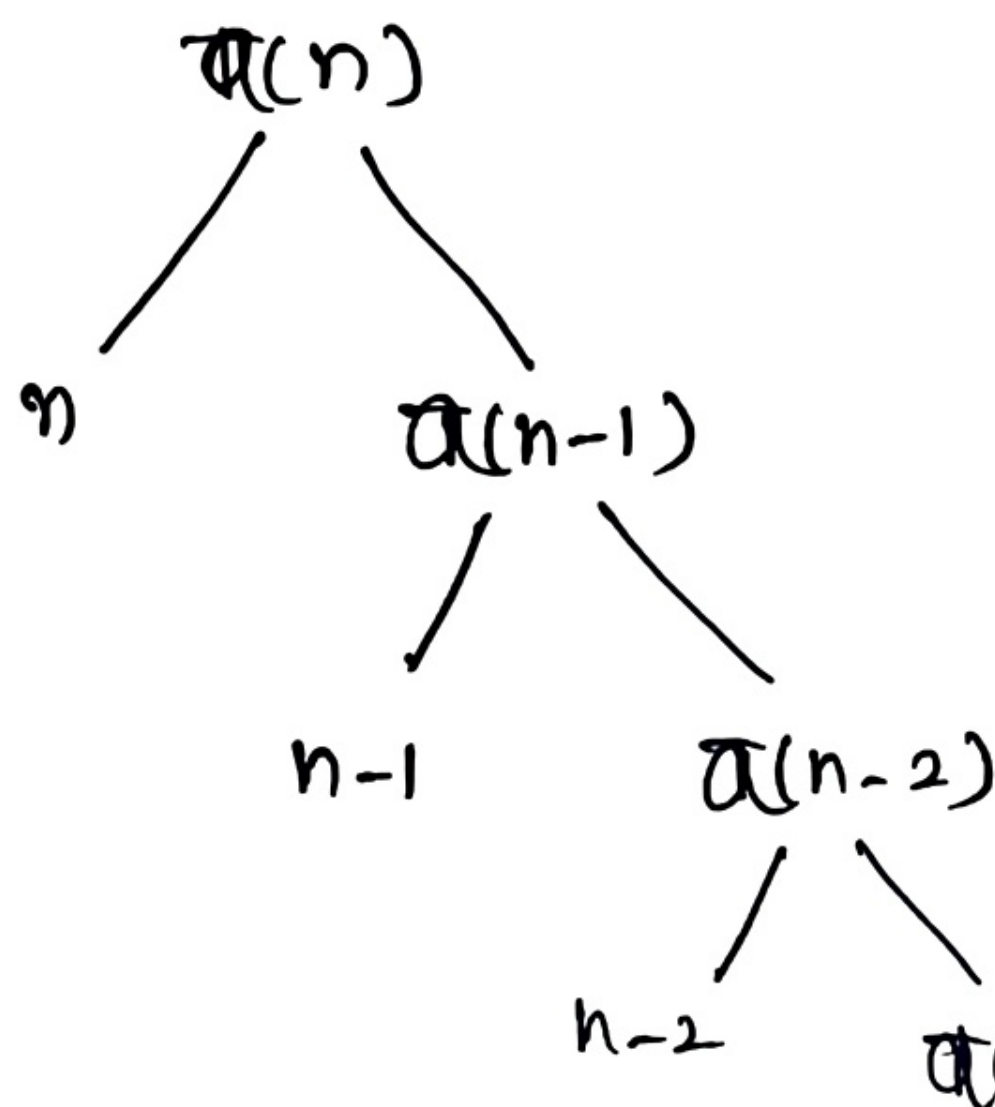
}

Based on Algorithm define recurrence relation

Recurrence Relation is!

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + 2 & n > 0 \end{cases}$$

Solve the recurrence relation using Recursion tree



$$\therefore T(n) = 0 + 1 + 2 + 3 + \dots + (n-1) + n$$

$\therefore$  The recurrence grows by 1 for every  $\uparrow$ se in  $n$ ,  
and grows linearly

$$\boxed{T(n) = O(n)}$$

3) MASTER'S THEOREM METHOD

Consider the following recurrence relation of form

$$T(n) = aT(n/b) + f(n) \text{ where } n \geq d \text{ \& } a$$

$d$  is some constant.

(or)

$$T(n) \neq aT(n/b) + f(n)$$

$\therefore$  Master theorem can be stated for efficiency analysis as.

\* If  $f(n)$  is  $\Theta(n^d)$  where  $d \geq 0$  in the recurrence relation then,

$$1. T(n) = \Theta(n^d) \text{ if } a < b^d$$

$$2. T(n) = \Theta(n^d \log n) \text{ if } a = b$$

$$3. T(n) = \Theta(n^{\log_b a}) \text{ if } a > b^d$$

Eg:  $T(n) = 4T(n/2) + n$  & solve by using Master method.

Sol  $T(n) = 4T(n/2) + n.$

$\therefore$  The recurrence relation form is

$$T(n) = aT(n/b) + f(n)$$

Here

$$a = 4 \quad b = 2 \quad f(n) = n$$

$$\text{Here } d \geq 0 \quad d = 1$$

Case 1:

$$a < b^d$$

$$4 < 2^1$$

$$4 < 2 \text{ [f]}$$

Case 2:

$$a = b$$

$$4 = 2 \text{ [f]}$$

Case 3:

$$a > b^d$$

$$4 > 2^1$$

$$4 > 2 \text{ [True]}$$

$\therefore$  case 3 is applied for the equation

$$T(n) = O(n^{\log b^a})$$

$$= O(n^{\log 2^4})$$

$$= O(n^{\log 2^2})$$

$$= O(n^2)$$

$$\boxed{T(n) = O(n^2)}$$

Time complexity for the given recurrence relation is  $O(n^2)$

## ANOTHER VARIATION OF MASTER THEOREM

\* For the Recurrence Relation

$$T(n) = aT(n/b) + f(n) \text{ if } n \geq d$$

1. If  $f(n)$  is  $O(n^{\log_b a - \epsilon})$ , then  $T(n) = \Theta(n^{\log_b a})$   
 $\forall \epsilon > 0$

2. If  $f(n)$  is  $\Theta(n^{\log_b a} \log^k n)$ , then  $k = 0, 1, 2, \dots$

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

3. If  $f(n)$  is  $\Omega(n^{\log_b a + \epsilon})$  then  $T(n)$  is  $\Theta(f(n))$

Eg:  $T(n) = 2T(n/2) + n \log n$

Sol Recurrence relation formula is:

$$T(n) = aT(n/b) + f(n) \quad n \geq d$$

Hence,  $a = 2$ ,  $b = 2$   $f(n) = n \log n$

Here we should put  $f(n) = n \log n$  in the above cases of master theorem.

Case 1:  $f(n) = O(n^{\log_b a - \epsilon}) \quad \epsilon > 0$

$$= O(n^{\log_2 2 - 1}) = O(n^0) = O(1)$$

$$f(n) = O(1)$$

$\therefore f(n)$  is not equal to  $n \log n$

Case 2:

$$f(n) = O(n^{\log_b a} \log^k n)$$

$$= O(n^{\log_2^2} \log^1 n)$$

$$k=1$$

$$f(n) = O(n \log n)$$

$\therefore$  function proved in case 2 is valid for defining Time complexity.

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

$$= \Theta(n^{\log_2^2} \log^2 n)$$

$$\boxed{T(n) = \Theta(n \log^2 n)}$$

eg:  $T(n) = T(n-1) + 1$   $T(0) = 1$

Apply forward

Let  $n=1$

$$T(1) = T(1-1) + 1$$

$$= T(0) + 1 \rightarrow \text{eq (1)}$$

Let  $n=2$

$$T(2) = T(2-1) + 1$$

$$= T(1) + 1 \rightarrow \text{eq (2)}$$

Sub eq (1) in eq (2)

$$T(2) = T(0) + 1 + 1$$

Let  $n=3$

$$T(3) = T(3-1) + 1$$

$$= T(2) + 1 \rightarrow \text{eq (3)}$$

Sub eq (3) in eq (2)

$$T(3) = T(0) + 1 + 1 + 1$$

⋮

$n$  times

$$T(n) = 1 + \underbrace{1 + 1 + 1 + \dots + 1}_{n \text{ times}} + (n-1) + n$$

$$\boxed{T(n) = O(n)}$$

Q:

- (1) Solve  $T(n) = 8T(n/2) + n^2$  by using master theorem
- (2) Solve  $T(n) = 9T(n/3) + n^3$  by using master theorem
- (3) Solve  $T(n) = T(n/2) + 1$  by using master theorem
- (4) Solve the following recurrence relation  
 $T(n) = T(n-1) + 1$  with  $T(0) = 0$  as initial

Condition & find big-oh notation.

5. Solve the recurrence relation

$$T(n) = 2T(n/2) + n$$

6. Solve the following recurrence relation

a)  $x(n) = x(n-1) + 5$  for  $n > 1$ ,  $x(1) = 0$

b)  $x(n) = 3x(n-1)$  for  $n > 1$ ,  $x(1) = 4$   $x(n) = 4 \cdot 3^{n-1}$

c)  $x(n) = x(n/3) + 1$  for  $n > 1$   $x(1) = 1$  (solve for  $n = 3^k$ )  
 $x(n) = 1 + \log_3 n$

d)  $x(n) = x(n/2) + n$  for  $n > 1$ ,  $x(1) = 1$  (solve for  $n = 2^k$ ),  
 $x(n) = 2n - 1$